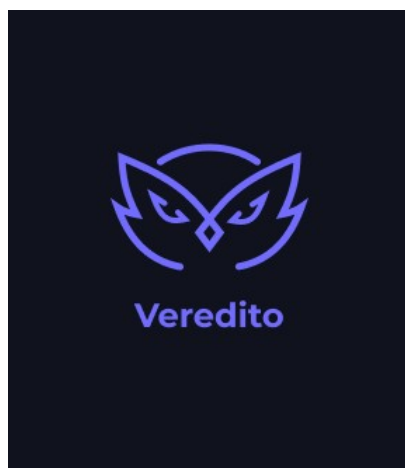




VEREDITO

Manual de Instalação

Sistema de Apoio à Decisão Judicial com Inteligência Artificial

São José dos Campos – SP
2025

RESUMO

Este documento descreve o procedimento de instalação e configuração do aplicativo Veredito em ambiente de desenvolvimento, homologação e produção. O manual abrange os requisitos de sistema, a configuração do frontend desenvolvido em Flutter e do backend baseado em Node.js com Docker, além das variáveis de ambiente necessárias para a integração com a API de inteligência artificial. O documento destina-se a desenvolvedores e administradores de sistema responsáveis pela implantação da solução.

Palavras-chave: instalação; configuração; Flutter; Docker; Node.js; variáveis de ambiente; manual técnico.

SUMÁRIO

Sumário

1 INTRODUÇÃO.....	4
2 REQUISITOS DO SISTEMA.....	4
2.1 Requisitos de Hardware.....	4
2.2 Requisitos de Software.....	4
3 OBTENÇÃO DO CÓDIGO-FONTE.....	5
4 INSTALAÇÃO E CONFIGURAÇÃO DO BACKEND.....	5
4.1 Visão Geral da Arquitetura do Backend.....	5
4.2 Configuração das Variáveis de Ambiente.....	6
4.2.1 Variável OPENAI_API_KEY.....	6
4.3 Build da Imagem Docker.....	6
4.4 Execução das Migrações de Banco de Dados.....	7
4.5 Inicialização do Servidor.....	7
4.6 Resumo dos Comandos do Backend.....	7
5 INSTALAÇÃO E CONFIGURAÇÃO DO FRONTEND.....	8
5.1 Visão Geral do Frontend.....	8
5.2 Instalação das Dependências.....	8
5.3 Configuração das Variáveis de Ambiente.....	8
5.4 Execução em Modo de Desenvolvimento.....	9
5.5 Execução em Ambiente de Homologação.....	9
5.6 Geração do APK de Produção.....	9
5.7 Instalação do APK no Dispositivo.....	10
5.8 Resumo dos Flavors Disponíveis.....	10
6 VERIFICAÇÃO DA INSTALAÇÃO.....	10
7 SOLUÇÃO DE PROBLEMAS.....	11
7.1 Backend não Responde na Porta 3000.....	11
7.2 Emulador Android não Consegue Acessar o Backend.....	11
7.3 Erro ao Executar flutter run.....	11
7.4 Erro de Autenticação com a OpenAI.....	11
8 REFERÊNCIAS.....	12

1 INTRODUÇÃO

O Veredito é um aplicativo móvel que combina um frontend desenvolvido em Flutter com um backend construído em Node.js, orquestrado por meio de contêineres Docker. A integração com modelos de linguagem natural da OpenAI permite a análise automatizada de documentos jurídicos e a sugestão de precedentes relevantes.

Este manual tem por objetivo guiar o responsável técnico em todas as etapas de instalação e configuração do sistema, desde a preparação do ambiente até a execução dos componentes em modo de desenvolvimento, homologação e produção. As instruções estão organizadas de forma sequencial e devem ser seguidas na ordem apresentada.

2 REQUISITOS DO SISTEMA

2.1 Requisitos de Hardware

Para executar o ambiente de desenvolvimento do Veredito de forma adequada, recomenda-se que a máquina do desenvolvedor possua, no mínimo, as seguintes configurações:

- Processador: Intel Core i5 (8ª geração ou superior) ou equivalente AMD;
- Memória RAM: 8 GB (16 GB recomendado para execução simultânea de emulador e Docker);
- Armazenamento: 20 GB de espaço livre em disco (SSD recomendado);
- Sistema operacional: Windows 10/11 (64 bits), macOS 12 (Monterey) ou superior, ou distribuição Linux baseada em Debian/Ubuntu.

2.2 Requisitos de Software

As ferramentas a seguir devem estar instaladas antes de prosseguir com a configuração do projeto:

- **Flutter SDK** 3.19 ou superior – disponível em <https://flutter.dev/docs/get-started/install>;
- **Dart SDK** (incluído no Flutter SDK);
- **Android Studio** ou VS Code com extensão Flutter instalada;

- **Android SDK** (API level 21 ou superior) para emulação ou build Android;
- **Docker** 24.0 ou superior – disponível em <https://docs.docker.com/get-docker/>;
- **Docker Compose** v2 ou superior (já incluso no Docker Desktop);
- **Git** 2.40 ou superior;
- **Chave de API da OpenAI** (obrigatória para o funcionamento das análises de IA) – obtenha em <https://platform.openai.com/api-keys>.

3 OBTENÇÃO DO CÓDIGO-FONTE

O código-fonte do Veredito está disponível em repositórios Git separados para o frontend e o backend. Execute os comandos abaixo no terminal para clonar os repositórios na máquina local:

```
# Clonar o repositório do frontend
git clone https://github.com/SkyFlyTeam/veredito-frontend

# Clonar o repositório do backend
git clone https://github.com/SkyFlyTeam/veredito-backend
```

4 INSTALAÇÃO E CONFIGURAÇÃO DO BACKEND

4.1 Visão Geral da Arquitetura do Backend

O backend do Veredito é uma aplicação Node.js gerenciada por meio do Docker Compose. A orquestração em contêineres garante portabilidade e isolamento de dependências, eliminando a necessidade de instalação manual de ferramentas como Node.js ou banco de dados na máquina host. Todas as dependências são resolvidas dentro dos contêineres.

4.2 Configuração das Variáveis de Ambiente

O projeto utiliza um arquivo .env para gerenciar as variáveis de configuração sensíveis. Um arquivo de modelo denominado .env-sample está disponível na raiz do repositório e deve ser copiado antes de qualquer execução. Execute o comando a seguir dentro do diretório do backend:

O parâmetro `--rm` garante que o contêiner temporário utilizado para a migração seja removido automaticamente ao final da execução. As migrações são aplicadas de forma incremental; portanto, o mesmo comando pode ser executado novamente após atualizações do sistema sem risco de perda de dados.

4.5 Inicialização do Servidor

Com as migrações aplicadas, inicie o servidor do backend com o seguinte comando:

```
docker compose run --rm --service-ports app
```

O parâmetro `--service-ports` expõe as portas configuradas no arquivo `docker-compose.yml` para o host, permitindo que o frontend e ferramentas externas se comuniquem com o backend. Por padrão, a aplicação é exposta na porta 3000.

Ao observar no terminal a mensagem indicando que o servidor está ouvindo na porta configurada, o backend estará pronto para receber requisições.

4.6 Resumo dos Comandos do Backend

A tabela a seguir resume a sequência completa de comandos necessários para inicializar o backend do Veredito a partir do zero:

Etapa	Comando
Copiar variáveis de ambiente	<code>cp .env-sample .env</code>
Build da imagem Docker	<code>docker compose build app</code>
Executar migrações	<code>docker compose run --rm app npm run migration:run</code>
Iniciar o servidor	<code>docker compose run --rm --service-ports app</code>

5 INSTALAÇÃO E CONFIGURAÇÃO DO FRONTEND

5.1 Visão Geral do Frontend

O frontend do Veredito é desenvolvido em Flutter, o framework multiplataforma do Google que permite gerar aplicativos para Android e iOS a partir de uma única base de código. O projeto utiliza o conceito de flavors para distinguir os ambientes de desenvolvimento (dev), homologação (stg) e produção (prod), garantindo configurações isoladas para cada cenário.

5.2 Instalação das Dependências

Após clonar o repositório do frontend, acesse o diretório do projeto e instale as dependências do Flutter com o comando:

```
cd <diretorio_do_frontend>
flutter pub get
```

O Flutter irá baixar e configurar automaticamente todos os pacotes listados no arquivo pubspec.yaml.

5.3 Configuração das Variáveis de Ambiente

O frontend consome a URL base da API do backend por meio de uma variável de ambiente. Crie um arquivo .env na raiz do projeto Flutter com o seguinte conteúdo:

```
API_URL="http://10.0.2.2:3000"
```

Nota: O endereço 10.0.2.2 é o alias padrão do emulador Android para acessar o localhost da máquina host. Para dispositivos físicos conectados via USB na mesma rede, substitua pelo endereço IP local da máquina que executa o backend (por exemplo, 192.168.1.100). Para iOS Simulator, utilize http://localhost:3000.

5.4 Execução em Modo de Desenvolvimento

Para executar o aplicativo em modo de desenvolvimento com hot reload ativo, utilize o comando abaixo. É necessário ter um emulador Android ou iOS em

execução, ou um dispositivo físico conectado via USB com depuração USB habilitada:

```
flutter run --flavor dev
```

O flavor dev carrega as configurações apropriadas para o ambiente de desenvolvimento, incluindo logs detalhados e ferramentas de diagnóstico.

5.5 Execução em Ambiente de Homologação

Para executar o aplicativo apontando para o ambiente de homologação (staging), utilize:

```
flutter run --flavor stg
```

5.6 Geração do APK de Produção

Para gerar o arquivo APK assinado destinado à distribuição em ambiente de produção, execute o seguinte comando:

```
flutter build apk --flavor prod --release
```

O processo de build compila o código Dart para código nativo ARM, aplica otimizações de produção e gera o arquivo APK na seguinte localização:

```
build/app/outputs/flutter-apk/app-prod-release.apk
```

5.7 Instalação do APK no Dispositivo

Após a geração do APK, instale-o no dispositivo Android de destino utilizando o comando ADB (Android Debug Bridge):

```
adb install build/app/outputs/flutter-apk/app-prod-release.apk
```

Alternativamente, transfira o arquivo APK para o dispositivo por qualquer meio (cabo USB, e-mail, serviço de armazenamento em nuvem) e instale-o manualmente a partir do gerenciador de arquivos do dispositivo. Certifique-se de que a opção "Instalar aplicativos de fontes desconhecidas" esteja habilitada nas configurações de segurança do dispositivo Android.

5.8 Resumo dos Flavors Disponíveis

Flavor	Comando	Descrição
dev	<code>flutter run --flavor dev</code>	Desenvolvimento local com hot reload e logs detalhados
stg	<code>flutter run --flavor stg</code>	Homologação, conectado ao ambiente de staging
prod	<code>flutter build apk --flavor prod --release</code>	Produção, build otimizado para distribuição

6 VERIFICAÇÃO DA INSTALAÇÃO

Após concluir a configuração de ambos os componentes, realize os seguintes testes para confirmar que o ambiente está funcionando corretamente:

1. Certifique-se de que o backend está em execução e ouvindo na porta 3000. Acesse `http://localhost:3000` em um navegador ou ferramenta como Postman e verifique se a API retorna uma resposta válida.
2. Inicie o emulador Android ou conecte um dispositivo físico.
3. Execute o frontend com `flutter run --flavor dev` e aguarde a compilação.
4. Ao abrir o aplicativo, a Tela de Splash deve ser exibida seguida da Tela de Login.
5. Realize um cadastro de teste e faça login para confirmar a comunicação entre frontend e backend.

7 SOLUÇÃO DE PROBLEMAS

7.1 Backend não Responde na Porta 3000

- Verifique se o Docker está em execução com o comando: `docker ps`.
- Confirme que não há outro processo utilizando a porta 3000: `lsof -i :3000` (Linux/macOS) ou `netstat -ano | findstr :3000` (Windows).
- Revise os logs do contêiner com: `docker compose logs app`.

7.2 Emulador Android não Consegue Acessar o Backend

- Certifique-se de que a variável `API_URL` está definida como `http://10.0.2.2:3000` no arquivo `.env` do frontend.
- Verifique se o firewall da máquina host não está bloqueando conexões na porta 3000.
- Para dispositivos físicos, substitua 10.0.2.2 pelo IP local da máquina host.

7.3 Erro ao Executar flutter run

- Execute flutter doctor para identificar dependências ausentes ou desatualizadas.
- Confirme que o Android SDK está instalado e que a variável `ANDROID_HOME` aponta para o diretório correto.
- Verifique se o emulador ou dispositivo físico está devidamente reconhecido com: `flutter devices`.

7.4 Erro de Autenticação com a OpenAI

- Confirme que a variável `OPENAI_API_KEY` no arquivo `.env` do backend contém uma chave válida e ativa.
- Verifique se a chave possui créditos disponíveis em <https://platform.openai.com/usage>.
- Reinicie os contêineres após qualquer alteração no arquivo `.env` com: `docker compose down && docker compose run --rm --service-ports app`.

8 REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: informação e documentação – referências – elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6024: informação e documentação – numeração progressiva das seções de um documento – apresentação. Rio de Janeiro: ABNT, 2012.

FLUTTER. Flutter documentation. Google LLC, 2024. Disponível em: <https://docs.flutter.dev>. Acesso em: maio 2025.

DOCKER INC. Docker documentation. Docker Inc., 2024. Disponível em: <https://docs.docker.com>. Acesso em: maio 2025.

OPENAI. API reference. OpenAI, 2024. Disponível em: <https://platform.openai.com/docs>. Acesso em: maio 2025.